

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ
ΥΠΟΛΟΓΙΣΤΩΝ

PART B WORKSHOP

SOFTWARE DEVELOPMENT IN C# AND RELIABILITY USING UNIT TESTS

TEAM MEMBER DETAILS

FULL NAME: ATHANASIOU VASILEIOS EVANGELOS

REGISTRATION NUMBER: 19390005

FULL NAME: THEOCHARIS GEORGIOS

REGISTRATION NUMBER: 19390283

FULL NAME: OMAR ALHAZ OMAR

REGISTRATION NUMBER: 19390010

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

CONTENTS

A. Development of a Function Library (DLL) for Personnel Management	4
1. bool ValidName(string Name)	4
1.1. Operation	4
1.2. Implementation	4
2. bool ValidPassword(string Password)	4
2.1. Operation	4
2.2. Implementation	5
3. EncryptPassword(string Password, ref string EncryptedPW)	5
3.1. Operation	5
3.2. Implementation	5
4. CheckPhone(string Phone, ref int TypePhone, ref string InfoPhone)	6
4.1. Operation	6
4.2. Implementation	6
5. InfoEmployee(Employee EmplX, ref int Age, ref int YearsOfExperience)	6
5.1. Operation	6
5.2. Implementation	6
6. int LiveInAthens(Employee Empls)	7
6.1. Operation	7
6.2. Implementation	7
B. Creating Test Cases (TestCases)	7
1. ValidName Test Case	7
2. ValidPassword Test Case	9
3. EncryptPassword Test Case	10
4. CheckPhone Test Case	11
5. InfoEmployee Test Case	13
6. LiveInAthens Test Case	15
C. Developing Unit Tests	17
1. ValidName Unit Test	17
2. ValidPassword Unit Test	19
3. EncryptPassword Unit Test	21
4. CheckPhone Unit Test	22
5. InfoEmployee Unit Test	25
6. LiveInAthens Unit Test	27
D. Audit Reports	31
1. Employee Name Error	31
1.1. Error ID	31

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

1.2. Name of failed test method	
31 1.3. Description of test performed	
31 1.4. Error Report that occurred	31 1.5.
Screenshot of Test Explorer	
32 2. Password Error	
32 2.1. Error ID	32
2.2. Name of failed test method	
32 2.3. Description of test performed	
32 2.4. Error Report that occurred	
32 2.5. Screenshot of Test Explorer	
33 3. Phone Error	
33 3.1. Error ID	33
3.2. Name of failed test method	33 3.3.
Description of test performed	33
3.4. Report of error encountered	
34 3.5. Screenshot of Test Explorer	
34 4. Employee Age Error	
34 4.1. Error ID	34
4.2. Name of failed test method	
35 4.3. Description of test performed	
35 4.4. Report of error encountered	
35 4.5. Screenshot of Test Explorer	
36 5. Error in Years of Service of an Employee	36
5.1. Error ID	36 5.2.
Name of the failed test method	36 5.3.
Description of the test performed	36
5.4. Report of the Error that occurred	
36 5.5. Screenshot of Test Explorer	37

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

A. Development of a Function Library (DLL) for Personnel Management

1. bool ValidName(string Name)

1.1. Operation

This function accepts a text as a parameter and returns a true or false value if the parameter may correspond to an employee's first and last name or not, respectively.

1.2. Implementation

The implementation assumptions of ValidName() are as follows:

- [1] The blank character ' ' must appear exactly once
- [2] We separate the first name from the last name based on the blank character ' ', where the same implementation assumptions apply to both
 - [2.1] The number of characters must be at least 3
 - [2.2] The number of characters must be a maximum of 15
 - [2.3] Must contain only letters
 - [2.4] The 1st letter must be capitalized
 - [2.5] Letters except the 1st must be lowercase
 - [2.6] The letters must all be Latin

2. bool ValidPassword(string Password)

2.1. Operation

This function accepts a text as a parameter and returns true or false if the parameter is an acceptable password. Password requirements: a. At least 12 characters, b. A combination of uppercase letters, lowercase letters, numbers and symbols (at least one character of each), c. The letters must be Latin characters, d. It must start with a capital letter and end with a number.

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

2.2. Implementation

The implementation assumptions of ValidPassword() are as follows:

- [1] The number of characters must be at least 12
- [2] The number of characters must be a maximum of 24
- [3] Character combination
 - [3.1] Must contain at least 1 capital letter
 - [3.2] Must contain at least 1 lowercase letter
 - [3.3] Must contain at least 1 digit
 - [3.4] Contain at least 1 special symbol
- [4] Do not contain escape characters
- [5] Letters must be Latin characters
- [6] Must start with a capital letter and end with a digit
 - [6.1] Must start with a capital letter
 - [6.2] Must end with a digit

3. EncryptPassword(string Password, ref string EncryptedPW)

3.1. Operation

This method accepts a password as a parameter and returns the encrypted password based on Caesar's Cipher, with the ASCII alphabet (128 characters, see [here](#)) and a shift of 5 places. This result is returned via the corresponding by ref parameter it accepts.

3.2. Implementation

The implementation assumptions of EncryptPassword() are as follows:

- [1] The code must be valid according to the [implementation assumptions](#) of the ValidPassword()

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

4. CheckPhone(string Phone, ref int TypePhone, ref string InfoPhone)

4.1. Operation

This method accepts a number as text and checks if it corresponds to a phone. The results are returned via the corresponding by ref parameters it accepts. Specifically, if it is a landline phone, it returns zero (0) as the phone type (TypePhone) and as the phone information (InfoPhone) the zone to which the phone number belongs (8 zones, [see here](#)). If it is a mobile phone, it returns the number one (1) as type and the mobile phone company as information ([see here](#)). Otherwise, if it is not a valid phone number, it returns the number type -1 and null in the information.

4.2. Implementation

The implementation assumptions of CheckPhone() are as follows:

- [1] Contain only numbers
- [2] The numbers must be exactly 10
- [3] Start at 2 if it is a fixed number
 - [3.1] Belong to a zone
- [4] Starts at 69 if it is a mobile phone
 - [4.1] Belong to a mobile phone company

5. InfoEmployee(Employee EmpIX, ref int Age, ref int YearsOfExperience)

5.1. Operation

This method accepts as parameters the details of an employee, and returns his age and years of service in the company. These results are returned through the corresponding by ref parameters it accepts.

5.2. Implementation

The implementation assumptions of InfoEmployee() are as follows:

- [1] Age must be between 18-70 years old
- [2] Date of hire must be from date of birth onwards
by 18 years to the current date

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

6. int LiveInAthens(Employee Empls)

6.1. Operation

This function returns the number of employees who live in Athens. This function calls the CheckPhone() function.

6.2. Implementation

The implementation assumptions of LiveInAthens() are as follows:

- [1] The landline number must start with "21" for residents of Athens
 - [1.1] Phone information should be "Athens - Piraeus Metropolitan Area" for Athens residents

B. Creating Test Cases (TestCases)

1. ValidName Test Case

id	Entry Condition	Equivalent price classes	
		Valid	Invalid
1	string[] subNames = Name.Split(' '); subNames.Length != 2	subNames.Length == 2	subNames.Length != 2
2	firstName = subNames[0]; lastName = subNames[1]; 2.1 firstName.Length		
< 3 OR	lastName.Length < 3 2.2	firstName.Length >= 3 AND lastName.Length >= 3	firstName.Length < 3 OR lastName.Length < 3
firstName.Length > 15 OR	lastName.Length > 15 2.3	firstName.Length <= 15 AND lastName.Length <= 15	firstName.Length > 15 OR lastName.Length > 15
NOT firstName.All(char.IsLetter) OR NOT lastName.All(char.IsLetter)		firstName.All(char.IsLetter) == true AND lastName.All(char.IsLetter) == true	firstName.All(char.IsLetter) == false OR lastName.All(char.IsLetter) == false
2.4 NOT char.IsUpper(firstName[0]) OR NOT char.IsUpper(lastName[0])		char.IsUpper(firstName[0]) == true AND char.IsUpper(lastName[0]) == true	char.IsUpper(firstName[0]) == false OR char.IsUpper(lastName[0]) == false
2.5 NOT	firstName.Substring(1).All(char.IsL (power) OR	firstName.Substring(1).All(ch ar.IsLower) == true AND lastName.Substring(1).All(ch	firstName.Substring(1).All(c har.IsLower) == false OR lastName.Substring(1).All(c

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

	NOT lastName.Substring(1).All(char.IsLower)	ar.IsLower) == true	har.IsLower) == false
2.6	char[] firstNameArray = firstName.ToCharArray(); char[] lastNameArray = lastName.ToCharArray(); Each Loop: NOT (firstNameArray[i] < 'A' OR firstNameArray[i] > 'Z') OR (firstNameArray[i] < 'a' OR firstNameArray[i] > 'z') OR Each Loop: NOT (lastNameArray[i] < 'A' OR lastNameArray[i] > 'Z') OR (lastNameArray[i] < 'a' OR lastNameArray[i] > 'z')	Each Loop : (firstNameArray[i] >= 'A' AND firstNameArray[i] <= 'Z') OR (firstNameArray[i] >= 'a' AND firstNameArray[i] <= 'z') AND Each Loop : (lastNameArray[i] >= 'A' AND lastNameArray[i] <= 'Z') OR (lastNameArray[i] >= 'a' AND lastNameArray[i] <= 'z')	Each Loop : (firstNameArray[i] < 'A' OR firstNameArray[i] > 'Z') OR (firstNameArray[i] < 'a' OR firstNameArray[i] > 'z') OR Each Loop : (lastNameArray[i] < 'A' OR lastNameArray[i] > 'Z') OR (lastNameArray[i] < 'a' OR lastNameArray[i] > 'z')

Per. El.	Test Input Data	Expected Results	
#id	Name	Flag	Hint
	Vasilis	false	firstNameAndSurname false
	Theoxaris	firstNameAndSurname false	moreThan3Chars
	About Alhaz	false	moreThan3Chars false lessThan15Chars
1	Vasilis At	false	lessThan15Chars false onlyLetters false
	Georgeeeeeeeeeee Theoxaris	onlyLetters false	firstLetterUpper false
	Omar Alhaaaaaaaaaz	firstLetterUpper false	restLettersLower false
	Vasilis123 Athanasiou	restLettersLower false	onlyLatin false
	George Theo_xaris	onlyLatin valid	Name true validName true
	omar Alhaz	validName true	firstNameAndSurname
2	Vasilis Athanasiou		
3	George Theoxaris		
4	Omar Alhaz		
5	Vasilis Athanasiou		
6	George Theocharis		
7	Omar Alhaz		
8	Vasilis Athanasiou		
9	George Theoxaris		
10 11 12 13 14 15 16 17	Name Vasilis Evangelos Athanasiou		

Hint	Message
firstNameAndSurname [1]	The blank character ' ' must appear exactly once
moreThan3Chars [2.1]	The number of characters must be at least 3
lessThan15Chars [2.2]	The number of characters must be at most 15
onlyLetters [2.3]	Must contain only letters
firstLetterUpper [2.5]	[2.4] The 1st letter must be capitalized
restLettersLower [2.6]	Letters except the 1st must be lowercase
onlyLatin	Letters must all be Latin
validName	Valid first and last name

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

2. ValidPassword Test Case

id	Entry Condition	Equivalent price classes	
		Valid	Invalid
1	Password.Length < 12	Password.Length >= 12	Password.Length < 12
2	Password.Length > 24	Password.Length <= 24	Password.Length > 24
3	Character combination		
3.1 NOT	Password.Any(char.IsUpper)	Password.Any(char.IsUpper) == true	Password.Any(char.IsUpper) == false
3.2 NOT	Password.Any(char.IsLower)	Password.Any(char.IsLower) == true	Password.Any(char.IsLower) == false
3.3 NOT	Password.Any(char.IsDigit)	Password.Any(char.IsDigit) == true	Password.Any(char.IsDigit) == false
3.4 NOT	Password.Any(symbol => "!@#\$%^&*()_+-=[]{} ;: '\",.<>/?".Contains(symbol))	Password.Any(symbol => "!@#\$%^&*()_+-=[]{} ;: '\",.<>/?".Contains(symbol)) == true	Password.Any(symbol => "!@#\$%^&*()_+-=[]{} ;: '\",.<>/?".Contains(symbol)) == false
4	Password.Any(char.IsWhiteSpace)	Password.Any(char.IsWhiteSpace) == false	Password.Any(char.IsWhiteSpace) == true
5	char[] passwordArray = Password.ToCharArray(); Each Loop: NOT (passwordArray[i] < 'A' OR passwordArray[i] > 'Z') OR (passwordArray[i] < 'a' OR passwordArray[i] > 'z')	Each Loop : (passwordArray[i] >= 'A' AND passwordArray[i] <= 'Z') OR (passwordArray[i] < 'a' AND passwordArray[i] > 'z')	Each Loop : (passwordArray[i] < 'A' OR passwordArray[i] > 'Z') OR (passwordArray[i] < 'a' OR passwordArray[i] > 'z')
6	It must start with a capital letter and end with a digit.		
6.1 NOT	char.IsUpper>Password[0])	char.IsUpper>Password[0]) == true	char.IsUpper>Password[0]) == false
6.2 NOT	char.IsDigit>Password[Password.Length - 1])	char.IsDigit>Password[Password.Length - 1]) == true	char.IsDigit>Password[Password.Length - 1]) == false

El.	Test Input Data	Expected Results	
#id	Password	Flag	Hint
1	Ako2	false	moreThan12Chars
2	Akkakbfpaoqweh!@#1224hhff1 !@#\$	false	lessThan24Chars
3	%^&*^%\$#@!	false	oneUpper + oneLower + oneDigit +

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

			one Symbol
4	george@12#po12	false	oneUpper
5	OMARGYPAS2l@13	false	oneLower
6	Ath@n@siouvasil	false	oneDigit
7	George2001theo13	false	oneSymbol
8	George!@#13 hf	false	noWhitespaces
9	ÿÿÿÿÿÿÿÿÿÿÿÿ@2024	false	onlyLatin
10	omara#Alhaz2001	false	startsWithUpper
11	Koaos1^klpoplkjq	false	endsWithDigit
12	Georgethe8o@#\$9	false	validPassword
13	Vasilisatha8no#6	true true	validPassword
14	Omar @Alhaz123	true	validPassword
PasswordError_2	Pao131908~123	true	oneSymbol

Hint	Message
moreThan12Chars	[1] The number of characters must be at least 12
lessThan24Chars	[2] The number of characters must be a maximum of 24
oneUpper	[3.1] Must contain at least 1 capital letter [3.2] Must
oneLower	contain at least 1 lowercase letter
oneDigit	[3.3] Must contain at least 1 digit
oneSymbol	[3.4] Contain at least 1 special symbol
noWhitespaces	[4] Must not contain escape characters [5]
onlyLatin	Letters must be Latin characters
startsWithUpper	[6.1] Must start with a capital letter
endsWithDigit	[6.2] Must end with a digit
validPassword	Valid password

3. EncryptPassword Test Case

id	Entry Condition	Equivalent price classes	
		Valid Invalid	
1	ValidPassword(Password)_	ValidPassword(Password d) == true d) == false	ValidPassword(Password

El.	Test Input Data	Expected Results	
#id 1	Password	EncryptedPW	Hint
	omar24323	null	invalidPassword
	willys12345	null	invalidPassword
2	theodos123	null	invalidPassword
3	Om	null	invalidPassword
4	Tigrhs_sdfdssdfsdfsdfsdf@16723	null	invalidPassword
5	Troussas_123	null	invalidPassword
6 7	G.Theo_@13223	L3YmjtdE68778	validPassword + validEncryption
8	Makh\$_@18923	Rfpm)dE6=>78	validPassword + validEncryption
9	Tango_@12093	YfsltdE675>8	validPassword + validEncryption
10	Lanaras_@16723	QfsfwfxdE6;<78	validPassword + validEncryption
11	George_!12525	Ljtwljd&67:7:	validPassword

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

			+ validEncryption
12	Krouska_123456	Pwtzxpfd6789;;	validPassword + validEncryption

Hint	Message
invalidPassword	[1] Password must be valid according to the implementation assumptions of ValidPassword() Valid password
validPassword	
validEncryption	Proper encryption

4. CheckPhone Test Case

id	Entry Condition	Equivalent price classes	
		Valid	Invalid
1 NOT	Phone.All(char.IsDigit)	Phone.All(char.IsDigit) == true	Phone.Length == 10 Phone.Length != 10 Phone.StartsWith("2") ==
2	Phone.Length != 10	Phone.StartsWith("2") == true	Phone[1] >= '1' Phone[1] < 1 AND
3	Phone.StartsWith("2")	Phone[1] <= '8'	
3.1	Phone[1] >= '1' AND Phone[1] <= '8'		Phone[1] > 8
4	Phone.StartsWith("69")	Phone.StartsWith("69") == true	Phone[2] == 0
4.1	Phone[2] == 0 OR (Phone[2] >= 3 AND Phone[2] <= 5) OR (Phone[2] >= 7 AND Phone[2] <= 9)	Phone[2] != 0 OR AND (Phone[2] >= 3 AND Phone[2] <= 5) OR AND (Phone[2] >= 7 AND Phone[2] <= 9)	(Phone[2] < 3 OR Phone[2] >= 5) (Phone[2] < 7 OR Phone[2] >= 9)

El.	Test Input Data	Expected Results		
#id	Phone	Type Phone	Phone Info	Hint
1	210-124567	-1	null	onlyDigits
2	690A49898c	-1	null	onlyDigits
3	22314	-1	null	digits10
4	69312	-1	null	digits10
5	23444444444444	-1	null	digits10
6	69988888888888	-1	null	digits10
7	140124567	-1	null	startsWith2HomePhone
8	2001010101	-1	null	belongsToZone
9	2101010101	0	Metropolitan Athens Area - Piraeus	validHomePhone + zone1
10	2201010101	0	Eastern Central Greece, Attica, Aegean Islands	validHomePhone + zone2
11	2301010101	0	Central Macedonia	validHomePhone + zone3

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

12	2401010101	0	Thessaly, Western Macedonia	validHomePhone + zone4
13	2501010101	0	Thrace, Eastern Macedonia	validHomePhone + zone5
14	2601010101	0	Epirus, Western Central Greece, Western Peloponnese, Ionian Islands	validHomePhone + zone6
15	2701010101	0	Eastern Peloponnese, Kythera	validHomePhone + zone7
16	2801010101	0	Crete	validHomePhone + zone8
17	2901010101	-1	null	belongsToZone
18	5971476763	-1	null	startsWith69MobilePhone
19	6900101010	1	Nova	validMobilePhone + nova
20	6910101010	-1	null	belongsToMobileDataComp
21	6920101010	-1	null	belongsToMobileDataComp
22	6930101010	1	Nova	validMobilePhone + nova
23	6940101010	1	Vodafone	validMobilePhone + vodafone
24	6950101010	1	Vodafone	validMobilePhone + vodafone
25	6960101010	-1	null	belongsToMobileDataComp
26	6970101010		Cosmote	validMobilePhone + cosmote
27	6980101010	1 1	Cosmote	validMobilePhone + cosmote
28	6990101010		Nova	validMobilePhone + nova onlyDigits
PhoneError_3	210 28 12 967	1 0	Metropolitan Athens Area – Piraeus	

Hint	Message
onlyDigits	[1] Contain only numbers
digits10	[2] The numbers must be exactly 10
startsWith2HomePhone	[3] Start at 2 if it is a fixed number
belongsToZone	[3.1] Belong to a zone
validHomePhone	Valid landline phone
zone1	with the Athens-Piraeus Metropolitan Area zone
zone2	with zone Eastern Central Greece, Attica, Aegean Islands
zone3	with Central Macedonia zone
zone4	with zone Thessaly, Western Macedonia
zone5	with zone Thrace, Eastern Macedonia
zone6	with zone Epirus, Western Central Greece, Western Peloponnese, Ionian Islands
zone7	with zone Eastern Peloponnese, Kythira
zone8	with zone Crete
startsWith69MobilePhone	[4] Starts at 69 if it is a mobile phone
belongsToMobileDataComp	[4.1] Belong to a mobile phone company
validMobilePhone	Valid mobile phone
new	with mobile phone company Nova with
cosmote	mobile phone company Cosmote with mobile
vodafone	phone company Vodafone

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

5. InfoEmployee Test Case

id	Entry Condition	Equivalent price classes	
		Valid	Invalid
1	EmplX.Birthday >= Today - 70 AND EmplX.Birthday <= Today - 18	EmplX.Birthday >= Today AND EmplX.Birthday <= Today - 18	EmplX.Birthday < Today - 70 - 70 OR EmplX.Birthday > Today - 18 - 18
2	EmplX.HiringDate >= EmplX.Birthday.AddYears(18) AND EmplX.HiringDate <= DateTime.Today	EmplX.HiringDate >= EmplX.HiringDate < EmplX.Birthday.AddYears(18) s(18) AND OR EmplX.HiringDate <= EmplX.HiringDate > DateTime.Today	EmplX.Birthday.AddYear DateTime.Today

Employee	Name	Home Phone	Mobile Phone	Birthday	Hiring Date
empl1	George Theocharis	2102322751	6998843565	2001-08-23	2022-05-08
empl2	Panagiotis Petropoulos	2102887987	6975522693	1985-04-03	2010-06-15
empl3	Vasilis Athanasiou	2201010101	6980101010	1999-07-15	2020-08-15
empl4	Evangelos Cement	2701010101	6990101010	1990-03-04	2015-06-03
empl5	Omar Alhaz	2301010101	6970101010	2000-02-01	2020-05-06
empl6	Akrivh Krouska	2301110101	6970101011	2000-02-01	2020-05-06
empl7	Pantelhs Tatsis	2601010101	6975522897	2001-01-08	2020-01-05
empl8	Aristotelis Sifakis	2601010801	6973298777	2001-01-07	2019-05-08
empl9	Antonis Kokkinos	2102322751	6978899987	2001-02-13	2021-05-02
empl10	Katerina Vossoy	2105042087	6987304875	2001-07-11	2022-09-01
empl11	Athina Nikolaou	2700101010	6975588489	1997-02-27	2023-05-22
empl12	Aggelikh Panou	2401010101	6954848497	1998-04-30	2019-06-07
empl13	Hlias Konstantinidis	2701010102	6935660423	1950-09-25	1970-04-08
empl14	George Gomez	2501010101	6990101010	2007-12-01	2023-08-01
empl15	George Santos	2105000000	6988888888	1990-04-29	2014-06-01
empl16	Antonis Varvaris	2300000000	6955555555	2000-01-10	2018-05-06
empl17	Xristos Pilarinos	2105555555	6987451235	1999-05-21	2019-06-30
emplfault1	Aggelos Skandalis	2501010102	6988558589	1953-05-30	1977-02-03
emplfault2	Dimitris Siametis	2104588989	6999999999	2006-01-02	2024-01-01

El.	Test Input Data	Expected Results		
#id	Employee	Age	Years Of Experience	Hint
1	empl1	24	1	validAge + "22" + validXpYears + "1"
2	empl2	38	13	validAge + "38" + validXpYears + "13"
3	empl3	24	3	validAge + "24" + validXpYears + "3"
4	empl4	33	8	validAge + "33" + validXpYears + "8"

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

5	empl5	24	3	validAge + "24" + validXpYears + "3"
6	employee6	24	3	validAge + "24" + validXpYears + "3"
7	empl7	23	4	validAge + "23" + validXpYears + "4"
8	empl8	23	4	validAge + "23" + validXpYears + "4"
9	empl9	23	2	validAge + "23" + validXpYears + "2"
10	empl10	22	1	validAge + "22" + validXpYears + "1"
11	empl11	26	0	validAge + "26" + validXpYears + "0"
12	empl12	25	4	validAge + "25" + validXpYears + "4"
13	empl13	-1	53	ageBetween18And70
14	empl14	-1	-1	youngerThan18ForHiring
15	empl15	33	9	validAge + "33" + validXpYears + "9"
16	empl16	24	5	validAge + "24" + validXpYears + "5"
17	empl17	24	4	validAge + "24" + validXpYears + "4"
AgeError_4	emplFault1	70	47	ageBetween18And70
YearsOfXpError_5	emplFault2	18	0	youngerThan18ForHiring

Hint	Message
ageBetween18And70	[1] Age must be between 18-70 years old
youngerThan18ForHiring	[2] Date of hire must be from date of birth plus 18 years to current date
validAge	Valid age : Valid
validXpYears	years of service:

6. LiveInAthens Test Case

id	Entry Condition	Equivalent price classes	
		Valid	Invalid
1	CheckPhone(emp.HomePhone, ref.Typephone, ref.InfoPhone)	TypePhone == 0 TypePhone != 0 AND AND InfoPhone != null InfoPhone == null InfoPhone ==	InfoPhone != "Metropolitan Area of Metropolitan Area of
1.1	InfoPhone == "Metropolitan Area of Athens - Piraeus	Athens - Piraeus Athens - Piraeus	

Employee List empls1				
Name	Home Phone	Mobile Phone	Birthday	Hiring Date
George Theoxaris	2202625345	6971345456	2001-08-23	2023-07-03
Vasilis Athanasiou	2302324901	6981314145	2001-03-19	2024-01-02
Omar Alhaz	2401234681	6945678789	2001-03-24	2023-12-20

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

Employee List empls2				
Name	Home Phone	Mobile Phone	Birthday	Hiring Date
Tom Hanks	2102625345	6971345456	1959-07-09	1989-06-09
Leonardo DiCaprio	2102324901	6981314145	1974-11-11	1995-03-15
Meryl Streep	2801234681	6945678789	1955-06-22	1975-05-05
Robert Downey	2102625345	6971345456	1965-04-04	1984-08-23
Johnny Depp	2102324901	6981314145	1963-06-09	1984-09-10
Angelina Jolie	2801234681	6945678789	1975-06-04	1994-10-12
Chris Hemsworth	2102324901	6981314145	1983-08-11	2002-03-18

Employee List empls3				
Name	Home Phone	Mobile Phone	Birthday	Hiring Date
Brad Pitt	2702625345	6971345456	1963-12-18	1987-05-20
Jennifer Aniston	2602324901	6981314145	1969-02-11	1992-07-15
Tom Cruise	2801234681	6945678789	1962-07-03	1981-10-10
George Clooney	2102625345	6971345456	1961-05-06	1982-09-22
Julia Roberts	2102324901	6981314145	1967-10-28	1988-07-15
Denzel Washington	2801234681	6945678789	1954-12-28	1978-06-12
Scarlett Johansson	2102625345	6971345456	1984-11-22	2003-05-10
Robert Pattinson	2801234681	6945678789	1986-05-13	2004-11-07

Employee List empls4				
Name	Home Phone	Mobile Phone	Birthday	Hiring Date
George Theoxaris	2102625345	6971345456	2001-08-23	2023-07-03
Vasilis Athanasiou	2102324901	6981314145	2001-03-19	2024-01-02
Omar Alhaz	2801234681	6945678789	2001-03-24	2023-12-20
Christos Troussas	2131414567	6941296963	1994-11-15	2024-01-01
Akrivi Krouska	2215678901	6901234345	1998-05-19	2023-12-20
John Vogiatzis	2801234681	6945678789	2001-03-24	2023-12-20
George Meletiou	2461345679	6931345450	1980-06-04	2023-11-03
Panagiotis Giannakopoulos	2151234510	6901234342	1978-06-10	2024-02-20
Stavros Fatouros	2612828456	6951616987	1990-01-24	2023-03-11

Employee List empls5				
Name	Home Phone	Mobile Phone	Birthday	Hiring Date
Bartholomew Dragowski	2145678901	6905678901	1997-08-19	2024-01-19
Tin Jedvaj	2156789012	6935678901	1995-11-28	2023-07-14
Filip Mladenovic	2167890123	6995678901	1991-08-15	2023-07-01
Giannis Kotsiras	2178901234	6945678901	1992-12-16	2021-07-01
Willian Arao	2189012345	6955678901	1992-03-12	2023-08-17
Carlos Zeca	2190123456	6975678901	1988-08-21	2023-07-01
Bernard Duarte	2101234567	6985678901	1992-09-08	2022-08-21
Anastasios Bakasetas	2112345678	6935678901	1993-06-28	2024-01-22
Adam Cerin	2123456789	6945678901	1999-07-16	2022-07-02
Sebastian Palacios	2134567890	6975678901	1992-01-20	2021-08-31
Fotis Ioannidis	2145678901	6985678901	2000-01-10	2020-08-10

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

Employee List empls6				
Name	Home Phone	Mobile Phone	Birthday	Hiring Date
George Theoxaris	2102625345	6971345456	2001-08-23	2023-07-03
Vasilis Athanasiou	2402324901	6981314145	2001-03-19	2024-01-02
Omar Alhaz	2501234681	6945678789	2001-03-24	2023-12-20
Dimitra Papadopoulou	2604567890	6934567890	2000-01-15	2023-06-20
Thanasis Papanicolaou	2705678901	6945678901	1999-11-28	2019-03-05
Eirini Nikolaou	2806789012	6956789012	2001-09-08	2023-08-12
Panagiotis Georgopoulos	2107890123	6907890123	2000-07-22	2019-01-01

Employee List empls7				
Name	Home Phone	Mobile Phone	Birthday	Hiring Date
Despina Vandi	2201234567	6931234567	1964-07-22	1998-10-01
Sakis Rouvas	2102345678	6942345678	1972-01-05	1990-05-15
Kostas Martakis	2703456789	6953456789	1974-09-14	2000-12-30
Elena Paparizou	2804567890	6944567890	1982-01-31	2010-02-20
Giorgos Liagas	2605678901	6975678901	1974-10-26	2001-07-10

Per. El.	Test Input Data	Expected Results	
#id	List Of Employees	Flag	Hint
1	empls1	"0/3"	numAthensEmps + 0
2	employees2	5	numAthensEmps + "5/7"
3	empls3	3	numAthensEmps + "3/8"
4	empls4	4	numAthensEmps + "3/9"
5	empls5	11	numAthensEmps + "11/11"
6	empls6	2	numAthensEmps + "3/7"
7	empls7	1	numAthensEmps + "1/5"

Hint	Message
numAthensEmps	Employees residing in Athens:

C. Developing Unit Tests (UnitTests)

1. ValidName Unit Test

```
[TestMethod]
public void TestMethodValidName() {

    // Implementation assumptions
    firstnameAndSurname = " [1] The blank character ' ' must occur exactly once"; string
```


ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
= string lessThan15Chars = " " [2.1] The number of characters must be at least 3"; string moreThan3Chars
[2.2] The number of characters must be at most 15"; string onlyLetters = " [2.3] It must contain only
letters"; string firstLetterUpper = " [2.4] The 1st letter must be
uppercase"; string restLettersLower = " [2.5] The letters except the 1st must be
lowercase"; string onlyLatin = " [2.6] The letters must all be Latin"; string validName = " Valid
name";
```

```
// Create an object of the Personnel class of HRLib.dll that we want to test
```

```
HRLib.Personnel per = new HRLib.Personnel();
```

```
// Create Test Cases object [,] testcases = { // { id, //
```

	"FirstName", estimated return value of ValidName()	"Implementation acceptance"
	{ "1",	
	"Vasilis", false, { "2",	firstnameAndSurname },
	"Theoxaris", false, { "3",	firstnameAndSurname },
	"Om Alhaz", false, { "4",	moreThan3Chars },
	"Vasilis At", false, { "5",	moreThan3Chars },
	"Georgeeeeeeeeeee Theoxaris", { "6", false, lessThan15Chars }, false, false,	
	"Omar Alhaaaaaaaaaaaaaz", false,	lessThan15Chars },
{ "7",	"Vasilis123 Athanasiou", false,	onlyLetters },
{ "8",	"George Theo_xaris", false,	onlyLetters },
{ "9", "omar Alhaz", { "10",	false,	firstLetterUpper },
	"Vasilis Athanasiou", false,	firstLetterUpper },
{ "11",	"George Theoxaris", false,	restLettersLower },
{ "12",	"Omar AlhAz", restLettersLower }, "Vasilis Athanasiou", { "13", onlyLatin },	
	true,	
{ "14", onlyLatin }, "George Theocharis", true,		
	"Omar Alhaz", validName }, { "16", true,	
	"Vasilis Athanasiou", validName }, { "17",	
	"George Theoxaris", validName },	
{ "NameError_1", "Vasileios Evangelos Athanasiou", true, firstnameAndSurname } };		

```
// Initialize Test Cases index int i = 0; bool failed = false;
```

```
// Access and execute control cases
```

```
for (i = 0; i < testcases.GetLength(0); i++) // For
```

```
each test case , i.e. for each row i of the testcases array {
```

```
try
```

```
{ // Call Assert.AreEqual giving the test case elements as parameters,
```

```
// i.e. the corresponding elements of row i of the testcases table string Name
```

```
= (string)testcases[i, 1]; bool expectedValue
```

```
= (bool)testcases[i, 2]; // The value I expect the ValidName() method to return
```

```
for date i
```

```
bool actualValue = per.ValidName(Name); // The value returned by the ValidName() method for date i
```

```
Assert.AreEqual(expectedValue, actualValue); } catch
```

```
(Exception e) { // The
```

```
Test Case failed failed =
```

```
true; // We
```

```
record the Test Case that failed
```

```
Console.WriteLine("Failed Test Case: {0} \n \t Assumption: {1} \n \t Exception: {2} ", e.Message);
```

```
(string)testcases[i, 0], (string)testcases[i, 3],
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
// id, Implementation assumption violated or valid control message,
Exception message }

}

// In case a Test Case failed, throw exception if (failed)

Assert.Fail();
}
```

2. ValidPassword Unit Test

```
[TestMethod]
public void TestMethodValidPassword() { //
```

Implementation assumptions

```
moreThan12Chars = " [2] The " [1] The number of characters must be at least 12"; string
= " [3.1] Contain at least 1 number of characters must be at most 24"; string lessThan24Chars
1 lowercase letter"; uppercase letter"; string oneUpper = " [3.2] Contain at least
string oneDigit = " string oneLower = " [3.3] Contain at least 1 digit";
oneSymbol = " [4] [3.4] Contain at least 1 special symbol"; string
" [5] The letters must not contain escape characters"; string noWhitespaces =
startsWithUpper = " [6.1] Latin characters"; string onlyLatin = string
Must end with a Must start with a capital letter"; string endsWithDigit = " [6.2]
digit"; string validPassword = " Valid password";
```

```
// Create an object of the Personnel class of HRLib.dll that we want to test
```

```
HRLib.Personnel per = new HRLib.Personnel();
```

```
// Creating Test Cases
```

```
object[,] testcases = { //
```

{ id, //	"Code",	estimated return value of ValidPassword()	"Implementation acceptance"
{ "1",	"Ako2",	false, moreThan12Chars },	
{ "2",	"Akkakbpaoqweh!@#1224hhff1",	false, " !@#\$	lessThan24Chars },
{ "3",	%^&*&^%\$#@!",	false,	oneUpper + oneLower + oneDigit
+ oneSymbol},			
{ "4", "george@12#po12", { "5",		false,	oneUpper },
"OMARGYPAS2!@13", { "6",		false,	oneLower },
"Ath@n@siouvasil", { "7",		false,	oneDigit },
"George2001theo13", { "8", "George!@#13		false ,	oneSymbol },
hf" { "9", "yyyyyyyyyy@2024", { "10", ,		false,	noWhitespaces },
"omara#Alhaz2001", { "11", "Koaos1^klpoplkjg",		false,	onlyLatin },
{ "12", "Georgethe8o@\$9", { "13",		false,	startsWithUpper },
"Vasilisatha8no#6", { "14",		false,	endsWithDigit },
"Omar_@Alhaz123", {"PasswordError_2",		true,	validPassword },
"Pao131908~123", };		true,	validPassword },
		true,	validPassword },
		true,	oneSymbol }

```
// Initialize test cases pointer (Test Cases)
```

```
int i = 0;
```

```
bool failed = false;
```

```
// Access and execute control cases
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```

for (i = 0; i < testcases.GetLength(0); i++)
// For each test case (Test Case), i.e. for each row i of the testcases table
{
    try

    { // Call Assert.AreEqual giving the test case elements as parameters,
      // i.e. the corresponding elements of row i of the testcases table
      string Password = (string)testcases[i, 1]; bool
      expectedValue = (bool)testcases[i, 2]; // The value I expect the method to return
ValidPassword() for date i
      bool actualValue = per.ValidPassword(Password); // The value returned by the method
ValidPassword() for date i
      Assert.AreEqual(expectedValue, actualValue); } catch

    (Exception e) { // Test

    Case failed
    failed = true; //
    Record the Test Case that failed
    Console.WriteLine("Failed Test Case: {0} \n \t Assumption: {1} \n \t Exception: {2} ", (string)testcases[i, 0],
      (string)testcases[i, 3], e.Message); id, Implementation assumption that is violated or valid check
      message, Exception message

    }
}

// In case a Test Case failed, throw an exception.
if (failed)
    Assert.Fail();
}

```

3. EncryptPassword Unit Test

```

[TestMethod]
public void TestMethodEncryptPassword() {

    // Implementation assumptions
    string invalidPassword = " [1] The password must be valid according to the implementation assumptions of
ValidPassword()";
    string validPassword = " Valid password"; string validEncryption =
    " Valid encryption";

    // Create an object of the Personnel class of HRLib.dll that we want to test
    HRLib.Personnel per = new HRLib.Personnel();

    // Create Test Cases object [,] testcases = { // { id, //

```

	"Code",	estimated return value of EncryptPassword()	"Implementation acceptance" }
{ "1",	"omar24323",	null, invalidPassword },	
{ "2",	"villys12345",	null, invalidPassword }, null, invalidPassword },	
{ "3",	"theodosis123",		
{ "4",	"Om",	null, invalidPassword },	
{ "5",	"Tigrhs_sdfdssdfsdfsdf@16723",	null,	invalidPassword },
{ "6",	"Troussas_123", null, "G.Theo_@13223",		invalidPassword },
{ "7",	"L3YmjtdE68778",		validPassword +
validEncryption },			
{ "8",	"Makh\$_@18923",	"Rfpm)dE6=>78",	validPassword +
validEncryption },			

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```

    { "9",          "Tango_@12093",          "YfsltdE675>8",          validPassword + validEncryption
  },
  { "10",          "Lanaras_@16723",          "QfsfwfxdE6;<78",          validPassword +
validEncryption },
  { "11",          "George_!12525",          "Ljtwljd&67:7:",          validPassword + validEncryption
  },
  { "12",          "Krouska_123456",          "Pwtzxpfd6789:;",          validPassword +
validEncryption } };
```

```
// Initialize Test Cases index int i = 0; bool failed = false;
```

```
// Access and execute control cases
```

```
for (i = 0; i < testcases.GetLength(0); i++) // For
```

```
each test case , i.e. for each row i of the testcases array {
```

```
try
{
```

```
    string TestcasePW = (string)testcases[i, 1]; // The code of testcase i string
```

```
    ExpectedEnPW = (string)testcases[i, 2]; // The encrypted code of testcase i that
```

```
I'm waiting for EncryptPassword() to return.
```

```
// Declare and initialize ref method variables
```

```
string ActualEnPW = "";
```

```
per.EncryptPassword(TestcasePW, ref ActualEnPW);
```

```
// Call Assert.AreEqual giving the test case elements as parameters,
```

```
// i.e. the corresponding elements of row i of the testcases array
```

```
Assert.AreEqual(ExpectedEnPW, ActualEnPW); }
```

```
catch (Exception e)
```

```
{ // Test Case failed
```

```
    failed = true; //
```

```
    Record the Test Case that failed
```

```
    Console.WriteLine("Failed Test Case: {0} \n \t Assumption: {1} \n \t Exception: {2} ",
```

```
        (string)testcases[i, 0], (string)testcases[i, 3], e.Message); Implementation assumption
```

```
    // that is violated or valid check message, id, Exception message }
```

```
}
```

```
// If a Test Case failed, throw an exception. if (failed)
```

```
    Assert.Fail();
```

```
}
```

4. CheckPhone Unit Test

```
[TestMethod]
```

```
public void TestMethodCheckPhone() {
```

```
    // Implementation assumptions
```

```
    string onlyDigits = " [1] Contain only numbers"; string
```

```
    digits10 = " [2] The numbers must be exactly 10";
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```

startsWithHomePhone = string " [3] Start with 2 if it is a landline"; string
belongsToZone = " [3.1] Belongs to a zone"; string
validHomePhone = "Valid landline"; string zone1 = with zone
Athens - Piraeus, Metropolitan Area"; string zone2 = with zone Eastern Central
Greece, Attica, Aegean Islands"; string zone3 = with zone Central Macedonia"; string
zone4 = with zone Thessaly, Western Macedonia"; string
zone5 = with zone Thrace, Eastern Macedonia"; with zone Epirus,
Western Central Greece, Western Peloponnese, Ionian Islands";
string zone8 = " „ string zone6 = with zone Eastern Peloponnese, Kythira"; string zone7 = with zone Crete";
[4] Start with 69 if it is a mobile"; string
startsWith69MobilePhone = " [4.1] Belongs
string validMobilePhoneCompany; Nova belongsToMobileDataComp = " Valid mobile phone";
vodafone = mobile phone company "; string nova = string cosmote = string

```

```

"
"
„with mobile phone company Cosmote"; with
mobile phone company Vodafone";

```

// Create an object of the Personnel class of HRLib.dll that we want to test

```
HRLib.Personnel per = new HRLib.Personnel();
```

// Create Test Cases object [,] testcases = { // { id,

```
return // { "1",
```

	"Phone Number", "Implementation Acceptance" }	estimated return value	estimated price
onlyDigits },			
{ "2",	-1,	null,	
onlyDigits },			
{ "3",	"690A49898c",	-1,	null,
	"22314",	-1,	null,
			digits10
{ "4",	"69312",	-1,	null,
			digits10
{ "5",	"2344444444444444",	-1,	null,
digits10 },			
{ "6",	"6998888888888888",	-1,	null,
digits10 },			
{ "7", "140124567",		-1,	null,
startsWithHomePhone },			
{ "8", "2001010101", belongsToZone },		-1,	null,
{ "9", "2101010101",			
validHomePhone + zone1 }, { "10",		0,	"Metropolitan Area of Athens - Piraeus",
"2201010101",		0,	"Eastern Central Greece, Attica, Aegean
Islands", validHomePhone + zone2 }, { "11",		0,	"Central Macedonia",
"2301010101", validHomePhone +		0,	
zone3 }, { "12", "2401010101",		0,	"Thessaly, Western Macedonia",
validHomePhone + zone4 }, { "13",		0,	
"2501010101", validHomePhone		0,	"Thrace, Eastern Macedonia",
+ zone5 }, { "14", "2601010101", 0,			
			"Epirus, Western Central Greece, Western
Peloponnese, Ionian Islands", validHomePhone + zone6 }, { "15",			
"2701010101", 0, validHomePhone + zone7 }, { "16",			"Eastern Peloponnese, Kythira",
"2801010101", validHomePhone			
+ zone8 }, { "17", "2901010101",		0,	"Crete",
belongsToZone }, { "18",			
"5971476763", startsWith69MobilePhone },		-1,	null,
		-1,	null,

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

{ "19",	"6900101010",	1,	"Nova",
validMobilePhone + nova },			
{ "20", "6910101010",		-1,	null,
belongsToMobileDataComp },			
{ "21", "6920101010",		-1,	null,
belongsToMobileDataComp },			
{ "22", "6930101010", validMobilePhone		1,	"Nova",
+ nova }, { "23", "6940101010",			
validMobilePhone + vodafone }, { "24",		1,	"Vodafone",
"6950101010", validMobilePhone +			
vodafone }, { "25", "6960101010",		1,	"Vodafone",
belongsToMobileDataComp }, { "26",			
"6970101010", validMobilePhone +		-1,	null,
cosmote }, { "27", "6980101010",			
validMobilePhone + cosmote }, { "28",		1,	"Cosmonaut",
"6990101010", validMobilePhone +			
nova },		1,	"Cosmonaut",
		1,	"Nova",
{ "PhoneError_3", "210 28 12 967",		0,	"Metropolitan Area of Athens -
Piraeus", onlyDigits } };			

```
// Initialize Test Cases index int i = 0; bool failed = false;
```

```
// Access and execute control cases
```

```
for (i = 0; i < testcases.GetLength(0); i++) // For  
each test case , i.e. for each row i of the testcases array {
```

```
try  
{
```

```
    string TestcasePhone = (string)testcases[i, 1]; int  
    ExpectedTypePhone = (int)testcases[i, 2];
```

```
// The phone number of testcase
```

```
i // The phone type of testcase i that I expect to receive
```

```
CheckPhone() returns string
```

```
    ExpectedInfoPhone = (string)testcases[i, 3]; // The phone information of testcase i that
```

```
I'm waiting for CheckPhone() to return.
```

```
// Declare and initialize ref method variables
```

```
int ActualTypePhone = 100;
```

```
string ActualInfoPhone = "";
```

```
per.CheckPhone(TestcasePhone, ref ActualTypePhone, ref ActualInfoPhone);
```

```
// Call Assert.AreEqual giving the test case elements as parameters,
```

```
// i.e. the corresponding elements of row i of the testcases array
```

```
Assert.AreEqual(ExpectedTypePhone, ActualTypePhone);
```

```
Assert.AreEqual(ExpectedInfoPhone, ActualInfoPhone); }
```

```
catch (Exception e)
```

```
{ // Test Case failed
```

```
    failed = true; //
```

```
Record the Test Case that failed
```

```
Console.WriteLine("Failed Test Case: {0} \n \t Assumption: {1} \n \t Exception: {2} ", (string)testcases[i,
```

```
0], (string)testcases[i, 4], e.Message); id, Implementation assumption that is violated or
```

```
// valid check message,
```

```
Exception message }
```

```
}
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
// If a Test Case failed, throw an exception. if (failed)
```

```
    Assert.Fail();
}
```

5. InfoEmployee Unit Test

```
[TestMethod]
```

```
public void TestMethodInfoEmployee() {
```

```
    // Implementation assumptions
```

```
    string ageBetween18And70 = "[1] Age must be between 18-70 years old"; string
```

```
    youngerThan18ForHiring = "[2] Hiring date must be between birth date and current date by 18 years"; string
```

```
    validAge = "Valid age : "; string validXpYears = "Valid years of service : ";
```

```
    // Create an object of the Personnel class of HRLib.dll that we want to test
```

```
    HRLib.Personnel per = new HRLib.Personnel();
```

of Birth", "Date of Employment"	"Full Name", // Date	"Landline Telephone",	"Mobile Phone",	"Date
Employee empl1 = new Employee("George Theocharis", new DateTime(2001, 08, 23), new DateTime(2022, 05, 08));		"2102322751",	"6998843565",	
Employee empl2 = new Employee("Panagiotis Petropoulos", new DateTime(1985, 04, 03), new DateTime(2010, 06, 15));		"2102887987",	"6975522693",	
Employee empl3 = new Employee("Vasilis Athanasiou", new DateTime(1999, 07, 15), new DateTime(2020, 08, 15));		"2201010101",	"6980101010",	new
Employee empl4 = new Employee("Evangelos Tsimentas", new DateTime(1990, 03, 04), new DateTime(2015, 06, 03));		"2701010101",	"6990101010",	
Employee empl5 = new Employee("Omar Alhaz", new DateTime(2000, 02, 01), new DateTime(2020, 05, 06));		"2301110101",	"6970101010",	new
Employee empl6 = new Employee("Akrivh Krouska", new DateTime(2000, 02, 01), new DateTime(2020, 05, 06));		"2301110101",	"6970101011",	new
Employee empl7 = new Employee("Pantelhs Tatsis", new DateTime(2001, 01, 08), new DateTime(2020, 01, 05));		"2601010101",	"6975522897",	new
Employee empl8 = new Employee("Aristotelis Sifakis", new DateTime(2001, 01, 07), new DateTime(2019, 05, 08));		"2601010801",	"6973298777",	new
Employee empl9 = new Employee("Antonis Kokkinos", new DateTime(2001, 02, 13), new DateTime(2021, 05, 02));		"2102322751",	"6978899987",	new
Employee empl10 = new Employee("Katerina Vossoy", new DateTime(2001, 07, 11), new DateTime(2022, 09, 01));		"2105042087",	"6987304875",	new
Employee empl11 = new Employee("Athina Nikolaou", new DateTime(1997, 02, 27), new DateTime(2023, 05, 22));		"2700101010",	"6975588489",	new
Employee empl12 = new Employee("Aggelikh Panou", new DateTime(1998, 04, 30), new DateTime(2019, 06, 07));		"2401010101",	"6954848497",	new
Employee empl13 = new Employee("Hlias Konstantinidis", new DateTime(1950, 09, 25), new DateTime(1970, 04, 08));		"2701010102",	"6935660423",	new
Employee empl14 = new Employee("George Gomez", new DateTime(2007, 12, 01), new DateTime(2023, 08, 01));		"2501010101",	"6990101010",	new
Employee empl15 = new Employee("George Santos", new DateTime(1990, 04, 29), new DateTime(2014, 06, 01));		"2105000000",	"6988888888",	new
Employee empl16 = new Employee("Antonis Varvaris", new DateTime(2000, 01, 10), new DateTime(2018, 05, 06));		"2300000000",	"6955555555",	new
Employee empl17 = new Employee("Xristos Pilarinos", new DateTime(1999, 05, 21), new DateTime(2019, 06, 30));		"2105555555",	"6987451235",	new
Employee emplfault1 = new Employee("Aggelos Skandalis", new DateTime(1953, 05, 30), new DateTime(1977, 02, 03));		"2501010102",	"6988558589",	
Employee emplfault2 = new Employee("Dimitris Siametis", new DateTime(2006, 01, 02), new DateTime(2024, 01, 01));		"2104588989",	"6999999999",	new

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
// Create Test Cases object [] testcases = { // { id,
"Employee",

"Implementation Assumption" }          estimated return price,          estimated return price,

//          of InfoEmployee() in Age of InfoEmployee() in YearsOfExperience 1, validAge + "22"
{ "1", empl1, + validXpYears          22,
+ "1" },
{ "2", empl2, + validXpYears          38,          13,          validAge + "38"
+ "13" },
{ "3", empl3, + validXpYears          24,          3,          validAge + "24"
+ "3" },
{ "4", empl4, + validXpYears          33,          8,          validAge + "33"
+ "8" },
{ "5", empl5, + validXpYears          24,          3,          validAge + "24"
+ "3" },
{ "6", empl6, + validXpYears          24,          3,          validAge + "24"
+ "3" },
{ "7", empl7, + validXpYears          23,          4,          validAge + "23"
+ "4" },
{ "8", empl8, + validXpYears          23,          4,          validAge + "23"
+ "4" },
{ "9", empl9, + validXpYears          23,          2,          validAge + "23"
+ "2" },
{ "10", empl10, + validXpYears          22,          1,          validAge + "22"
+ "1" },
{ "11", empl11, + validXpYears          27,          0,          validAge + "26"
+ "0" },
{ "12", empl12, + validXpYears          25,          4,          validAge + "25"
+ "4" },
{ "13", empl13,
ageBetween18And70 },
{ "14", empl14,
youngerThan18ForHiring },
{ "15", empl15, + validXpYears          33,          9,          validAge + "33"
+ "9" },
{ "16", empl16, + validXpYears          24,          5,          validAge + "24"
+ "5" },
{ "17", empl17, + validXpYears          24,          4,          validAge + "24"
+ "4" },
{ "AgeError_4",          emplfault1,          70,          47,
ageBetween18And70 },
{ "YearsOfXpError_5", emplfault2,          18,          0,
youngerThan18ForHiring }

};

// Initialize Test Cases index int i = 0; bool failed = false;

// Access and execute control cases
for (i = 0; i < testcases.GetLength(0); i++) // For
each test case , i.e. for each row i of the testcases array {

    try
    {
        Employee TestcaseEmployee = (Employee)testcases[i, 1]; int          // The employee of testcase i //
        ExpectedAge = (int)testcases[i, 2];          The age of the employee of testcase i that
        I'm waiting for InfoEmployee() to return.
    }
}
```


ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```

    int ExpectedYearsOfExperience = (int)testcases[i, 3]; // The employee's years of service
    testcase i that I expect InfoEmployee() to return

    // Declare and initialize ref method variables
    int ActualAge = 0; int
    ActualYearsOfExperience = 100;

    // Call InfoEmployee using ref to return the ActualAge and ActualYearsOfExperience values //
as well as the date
    formatting FormattedDate per.InfoEmployee(TestcaseEmployee, ref
    ActualAge, ref ActualYearsOfExperience);

    // Call Assert.AreEqual giving the test case elements as parameters,
    // i.e. the corresponding elements of row i of the testcases table
    Assert.AreEqual(ExpectedAge, ActualAge);
    Assert.AreEqual(ExpectedYearsOfExperience, ActualYearsOfExperience); } catch

(Exception e) { // Test

Case failed
    failed = true; //
    Record the Test Case that failed
    Console.WriteLine("Failed Test Case: {0} \n \t Assumption: {1} \n \t Exception: {2} ", (string)testcases[i,
    0], (string)testcases[i, 4], e.Message); id, Implementation assumption violated or valid check
    // message, Exception message }

}

// If a Test Case failed, throw an exception. if (failed)

    Assert.Fail();
}

```

6. LiveInAthens Unit Test

```

[TestMethod]
public void TestMethodLiveInAthens() { //

Implementation assumptions
    numAthensEmps = "Employees residing in Athens: "; string

    // Create an object of the Personnel class of HRLib.dll that we want to test
    HRLib.Personnel per = new HRLib.Personnel();

    Employee[] empls1 = new Employee[]
    {
        "Name", "Landline Telephone", "Mobile Telephone", // "Hiring Date" "Date of Birth",
        new Employee("George Theoxaris", "2202625345", 23), new "6971345456", new DateTime(2001, 08,
        DateTime(2023, 07, 03)), new Employee("Vasilis
        Athanasiou", "2302324901", new DateTime(2024, 01, 02)), "6981314145", new DateTime(2001, 03, 19),
        new Employee("Omar Alhaz", "2401234681", "6945678789", new DateTime(2001, 03, 24),
        new DateTime(2023, 12, 20))
    };
    Employee[] empls2 = new Employee[] { //

    "First and Last Name", "Landline Phone", "Mobile Phone", "Hiring Date" "Date of Birth",

```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```

    new Employee("Tom Hanks", "2102625345", "6971345456", new DateTime(1959, 07, 09),
    new DateTime(1989, 06, 09)),
    new Employee("Leonardo DiCaprio", "2102324901", 11), new "6981314145", new DateTime(1974, 11,
    DateTime(1995, 03, 15)), new Employee("Meryl
    Streep", "2801234681", "6945678789", new DateTime(1955, 06, 22),
    new DateTime(1975, 05, 05)), new
    Employee("Robert Downey", new "2102625345", "6971345456", new DateTime(1965, 04, 04),
    DateTime(1984, 08, 23)),
    new Employee("Johnny Depp", "2102324901", "6981314145", new DateTime(1963, 06, 09),
    new DateTime(1984, 09, 10)), new
    Employee("Angelina Jolie", "2801234681", new "6945678789", new DateTime(1975, 06, 04),
    DateTime(1994, 10, 12)),
    new Employee("Chris Hemsworth", "2102324901", 11), new "6981314145", new DateTime(1983, 08,
    DateTime(2002, 03, 18)), };

```

```
Employee[] empls3 = new Employee[] { //
```

"First and Last Name", "Landline Phone", "Mobile Phone", "Hiring Date

"Date of Birth",

```

    new Employee("Brad Pitt", "2702625345", "6971345456", new DateTime(1963, 12, 18),
    new DateTime(1987, 05, 20)), new
    Employee("Jennifer Aniston", "2602324901", new "6981314145", new DateTime(1969, 02, 11),
    DateTime(1992, 07, 15)),
    new Employee("Tom Cruise", "2801234681", "6945678789", new DateTime(1962, 07, 03),
    new DateTime(1981, 10, 10)), new
    Employee("George Clooney", "2102625345", new "6971345456", new DateTime(1961, 05, 06),
    DateTime(1982, 09, 22)),
    new Employee("Julia Roberts", "2102324901", "6981314145", new DateTime(1967, 10, 28),
    new DateTime(1988, 07, 15)),
    new Employee("Denzel Washington", "2801234681", 28), new "6945678789", new DateTime(1954, 12,
    DateTime(1978, 06, 12)), new Employee("Scarlett
    Johansson", "2102625345", new DateTime(2003, 05, 10)), "6971345456", new DateTime(1984, 11, 22),
    new Employee("Robert Pattinson",
    "2801234681", "6945678789", new DateTime(1986, 05, 13),
    new DateTime(2004, 11, 07))

```

```
};
```

```
Employee[] empls4 = new Employee[] {
```

// "Name", "Date of Hiring" "Landline Telephone", "Mobile Telephone", "Date of Birth",

```

    new Employee("George Theoxaris", "2102625345", 08, 23), new "6971345456", new DateTime(2001,
    DateTime(2023, 07, 03)), new Employee("Vasilis
    Athanasiou", 03, 19), new DateTime(2024, "2102324901", "6981314145", new DateTime(2001,
    01, 02)), new Employee("Omar Alhaz", "2801234681",
    24), new DateTime(2023, 12, 20)), new Employee("Christos
    Troussas", 11, 15), new DateTime(2024, 01, 01)), "6945678789", new DateTime(2001, 03,
    new Employee("Akrivi Krouska", "2131414567", "6941296963", new DateTime(1994,
    "2215678901", 19), new DateTime(2023, 12, 20)), new
    Employee("Ioannis Vogiatzis", "2801234681", 24), new "6901234345", new DateTime(1998, 05,
    DateTime(2023, 12, 20)), new Employee("George
    Meletiou", 06, 04), new DateTime(2023, 11, 03)), new "6945678789", new DateTime(2001, 03,
    Employee("Panagiotis Giannakopoulos",
    "2151234510", "2461345679", "6931345450", new DateTime(1980,
    "6901234342", new
    DateTime(1978, 06, 10), new DateTime(2024, 02, 20)), new
    Employee("Stavros Fatouros", "2612828456", 01, 24), new "6951616987", new DateTime(1990,
    DateTime(2023, 03, 11)) };

```

```
Employee[] empls5 = new Employee[] {
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```
// "Name", "Date of Hiring", "Landline Telephone", "Mobile Telephone", "Date of Birth",
new Employee("Bartlomiej Dragowski", "2145678901", 08, 19), new "6905678901", new DateTime(1997,
DateTime(2024, 01, 19)), new Employee("Tin Jedvaj",
28), new DateTime(2023, 07, "2156789012", "6935678901", new DateTime(1995, 11,
14)), new Employee("Filip Mladenovic",
"2167890123", 15), new DateTime(2023, 07, 01)), new "6995678901", new DateTime(1991, 08,
Employee("Giannis Kotsiras", "2178901234", 16),
new DateTime(2021, 07, 01)), new Employee("Willian Arao", "6945678901", new DateTime(1992, 12,
"2189012345", 12), new DateTime(2023, 08, 17)),
new Employee("Carlos Zeca", "2190123456", 21), new "6955678901", new DateTime(1992, 03,
DateTime(2023, 07, 01)), new Employee("Bernard
Duarte", 09, 08), new DateTime(2022, 08, 21)), new "6975678901", new DateTime(1988, 08,
Employee("Anastasios Bakasetas", "2112345678",
06, 28), new DateTime(2024, 01, 22)), "2101234567", "6985678901", new DateTime(1992,
new Employee("Adam Cerin", "2123456789", 16), new
DateTime(2022, 07, 02)), new Employee("Sebastian Palacios", 01, "6935678901", new DateTime(1993,
20), new DateTime(2021, 08, 31)), new Employee("Fotis
Ioannidis", "2145678901", 10), new DateTime(2020, 08, 10)) }; "6945678901", new DateTime(1999, 07,
"2134567890", "6975678901", new DateTime(1992,
"6985678901", new DateTime(2000, 01,
```

```
Employee[] empls6 = new Employee[] {
```

```
"Full Name", // Date "Landline Telephone", "Mobile Telephone", "Date
of Birth", "Date of Hiring"
new Employee("George Theoxaris", "2102625345", 08, 23), new "6971345456", new DateTime(2001,
DateTime(2023, 07, 03)), new Employee("Vasilis
Athanasίου", 03, 19), new DateTime(2024, "2402324901", "6981314145", new DateTime(2001,
01, 02)), new Employee("Omar Alhaz", "2501234681",
24), new DateTime(2023, 12, 20)), new Employee("Dimitra
Papadopoulou", "2604567890", 01, 15), new "6945678789", new DateTime(2001, 03,
DateTime(2023, 06, 20)), new Employee("Thanasis Papanikolaou", "6934567890", new DateTime(2000,
"2705678901", 11, 28), new DateTime(2019, 03, 05)),
new Employee("Eirini Nikolaou", "2806789012", 8), new "6945678901", new DateTime(1999,
DateTime(2023, 08, 12)), new Employee("Panagiotis
Georgopoulos", "2107890123", 07, 22), new DateTime(2019, "6956789012", new DateTime(2001, 09,
01, 01)) }; "6907890123", new DateTime(2000,
```

```
Employee[] empls7 = new Employee[] {
```

```
"Full Name", "Landline Telephone", "Mobile Telephone", // Date of Birth", "Date
Date of Hiring"
new Employee("Despina Vandi", "2201234567", 07, 22), new "6931234567", new DateTime(1964,
DateTime(1998, 10, 01)), new Employee("Sakis
Rouvas", "2102345678", 05), new DateTime(1990, 05, 15)), new "6942345678", new DateTime(1972, 01,
Employee("Kostas Martakis", "2703456789", 09,
14), new DateTime(2000, 12, 30)), new Employee("Elena
Paparizou", "2804567890", 01, 31), new DateTime(2010,
02, 20)), new Employee("Giorgos Liagas", "2605678901", 26), "6944567890", new DateTime(1982,
new DateTime(2001, 07, 10)) }; "6975678901", new DateTime(1974, 10,
```

```
// Create Test Cases object [.] testcases =
```

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

```

{ //      "Employee List", estimated return value,      "Implementation acceptance"}
{ id, //      of LiveInAthens() 0,
  { "1",      empls1,      numAthensEmps + "0/3" },
  { "2",      empls2,      numAthensEmps + "5/7" },
  { "3",      empls3,      numAthensEmps + "3/8" },
  { "4",      empls4,      numAthensEmps + "3/9" },
  { "5",      empls5,      5,      numAthensEmps + "11/11" },
  { "6",      empls6,      numAthensEmps + "3/7" },
  { "7",      empls7,      3, 4, 11, 2, 1,      numAthensEmps + "1/5" }
};

// Initialize test cases pointer (Test Cases)
int i = 0;
bool failed = false;

// Access and execute control cases
for (i = 0; i < testcases.GetLength(0); i++)
// For each test case (Test Case), i.e. for each row i of the testcases table

{ try
  {
    Employee[] TestcaseEmps = (Employee[])testcases[i, 1];      // The list of employees
test case i
    int ExpectedAthensHabitants = (int)testcases[i, 2];      // The residents of Athens of testcase i who
I'm waiting for LiveInAthens() to return.
    int ActualAthensHabitants = per.LiveInAthens(TestcaseEmps); // The residents of Athens in the testcase
i returned by LiveInAthens()

    // Call Assert.AreEqual giving the test case elements as parameters,
    // i.e. the corresponding elements of row i of the testcases table
    Assert.AreEqual(ExpectedAthensHabitants, ActualAthensHabitants); } catch

    (Exception e) { // Test

    Case failed
    failed = true; //
    Record the Test Case that failed
    Console.WriteLine("Failed Test Case: {0} \n \t Assumption: {1} \n \t Exception: {2} ", (string)testcases[i,
    0], (string)testcases[i, 3], e.Message); Implementation assumption that is violated or
    //      message of valid id, control, Message of exception

    }
  }

// In case a Test Case failed, throw an exception.
if (failed)
  Assert.Fail();
}

```

D. Audit Reports

1. Employee Name Error

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

1.1. Error ID

NameError_1

1.2. Name of failed control method

TestMethodValidName

1.3. Description of the audit carried out

The check is performed to determine whether the textual content of the input data corresponds to an employee's name. In detail, if the textual content meets all the implementation assumptions of the ValidName() method mentioned [here](#), then it corresponds to a valid employee's name. Otherwise, if it violates at least one assumption, then it cannot correspond to a valid employee's name.

1.4. Report an Error that occurred

Per. El.	Test Input Data	Expected Results	
#id	Name	Flag	Hint
NameError_1	Vasileios Evangelos Athanasiou	true	firstnameAndSurname

Hint	Message
firstnameAndSurname [1]	The blank character ' ' must appear exactly once

According to the implementation assumptions of the ValidName() method mentioned [here](#), the input data with #id NameError_1, violates the assumption:

[1] The blank character ' ' must appear exactly once

The name of the test case with #id NameError_1 contains 2 blank characters ' ', which leads to an error. Essentially, the software prevents the insertion of a possible 2nd name of an employee, resulting in inaccuracy regarding the reliability of his/her information.

1.5. Screenshot of Test Explorer

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

Test Explorer

6 2 4

Test	Duration	Traits	Error Message
✖ UnitTestHRLib (6)	153 ms		
✖ HRLibUnitTest (6)	153 ms		
✖ UnitTestPersonnel (6)	153 ms		
✖ TestMethodCheckPhone	140 ms		Assert.Fail failed.
✔ TestMethodEncryptPassword	2 ms		
✖ TestMethodInfoEmployee	4 ms		Assert.Fail failed.
✔ TestMethodLiveInAthens	3 ms		
✖ TestMethodValidName	3 ms		Assert.Fail failed.
✖ TestMethodValidPassword	1 ms		Assert.Fail failed.

Test Detail Summary

✖ TestMethodValidName

Source: [UnitTestPersonnel.cs](#) line 12

Duration: 3 ms

Message:
Assert.Fail failed.

Stack Trace:
[UnitTestPersonnel.TestMethodValidName\(\)](#) line 82

Standard Output:
Αποτυχημένο Test Case: NameError_1
Παραδοχή: [1] Να υπάρχει ο χαρακτήρας κενό ' ' ακριβώς μία φορά
Εξαίρεση: Assert.AreEqual failed. Expected:<True>. Actual:<False>.

2. Password Error

2.1. Error ID

PasswordError_2

2.2. Name of failed control method

TestMethodValidPassword

2.3. Description of the audit carried out

The check is performed to determine whether the textual content of the input data corresponds to an acceptable password. Specifically, whether the textual content meets all the implementation assumptions of the ValidPassword() method mentioned above.

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

[here](#), then it corresponds to a valid password. Otherwise, if it violates at least one assumption, then it cannot correspond to an acceptable password.

2.4. Report an Error that occurred

Per. El.	Test Input Data	Expected Results	
#id	Password	Flag	Hint
PasswordError_2	Pao131908~123	true	oneSymbol

Hint	Message
oneSymbol	[3.4] Must contain at least 1 special symbol

According to the implementation assumptions of the ValidPassword() method mentioned [here](#), the input with #id PasswordError_2 violates the assumption:

[3.4] Contain at least 1 special symbol

The password of the test case with #id PasswordError_2 contains the special symbol ~, which leads to an error. Essentially, the software accepts as special symbols only the symbols ! @# \$ % ^ & * () _ + - = [] { } ; : " ' , . < > / ? that belong to this set. The symbol ~ does not belong to this set and with this assumption, the software limits the number of accepted special symbols that satisfy the assumption [3.4] that was violated in the above.

2.5. Screenshot of Test Explorer

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

Test Explorer

Test ▲ | Duration | Traits | Error Message

- ✖ UnitTestHRLib (6) | 153 ms
- ✖ HRLibUnitTest (6) | 153 ms
- ✖ UnitTestPersonnel (6) | 153 ms
 - ✖ TestMethodCheckPhone | 140 ms | Assert.Fail failed.
 - ✔ TestMethodEncryptPassword | 2 ms
 - ✖ TestMethodInfoEmployee | 4 ms | Assert.Fail failed.
 - ✔ TestMethodLiveInAthens | 3 ms
 - ✖ TestMethodValidName | 3 ms | Assert.Fail failed.
 - ✖ TestMethodValidPassword | 1 ms | Assert.Fail failed.

Test Detail Summary

✖ TestMethodValidPassword

Source: [UnitTestPersonnel.cs](#) line 86

Duration: 1 ms

Message:
Assert.Fail failed.

Stack Trace:
[UnitTestPersonnel.TestMethodValidPassword\(\)](#) line 157

Standard Output:
Αποτυχημένο Test Case: PasswordError_2
Παραδοχή: [3.4] Να περιέχει τουλάχιστον 1 ειδικό σύμβολο
Εξαίρεση: Assert.AreEqual failed. Expected:<True>. Actual:<False>.

3. Phone Error

3.1. Error ID

PhoneError_3

3.2. Name of failed control method

TestMethodCheckPhone

3.3. Description of the audit carried out

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

The check is performed to determine whether the textual content of the input data corresponds to an employee's phone number (either landline or mobile). In detail, if the textual content meets all the implementation assumptions of the `CheckPhone()` method mentioned [here](#), then it corresponds to a valid employee's phone number. Otherwise, if it violates at least one assumption, then it cannot correspond to a valid employee's phone number.

3.4. Report an Error that occurred

El.	Test Input Data	Expected Results		
#id	Phone	Type Phone	Phone Info	Hint
PhoneError_3	210 28 12 967	0	Athens Metropolitan Area - Piraeus	onlyDigits

Hint	Message
onlyDigits	[1] Contain only numbers

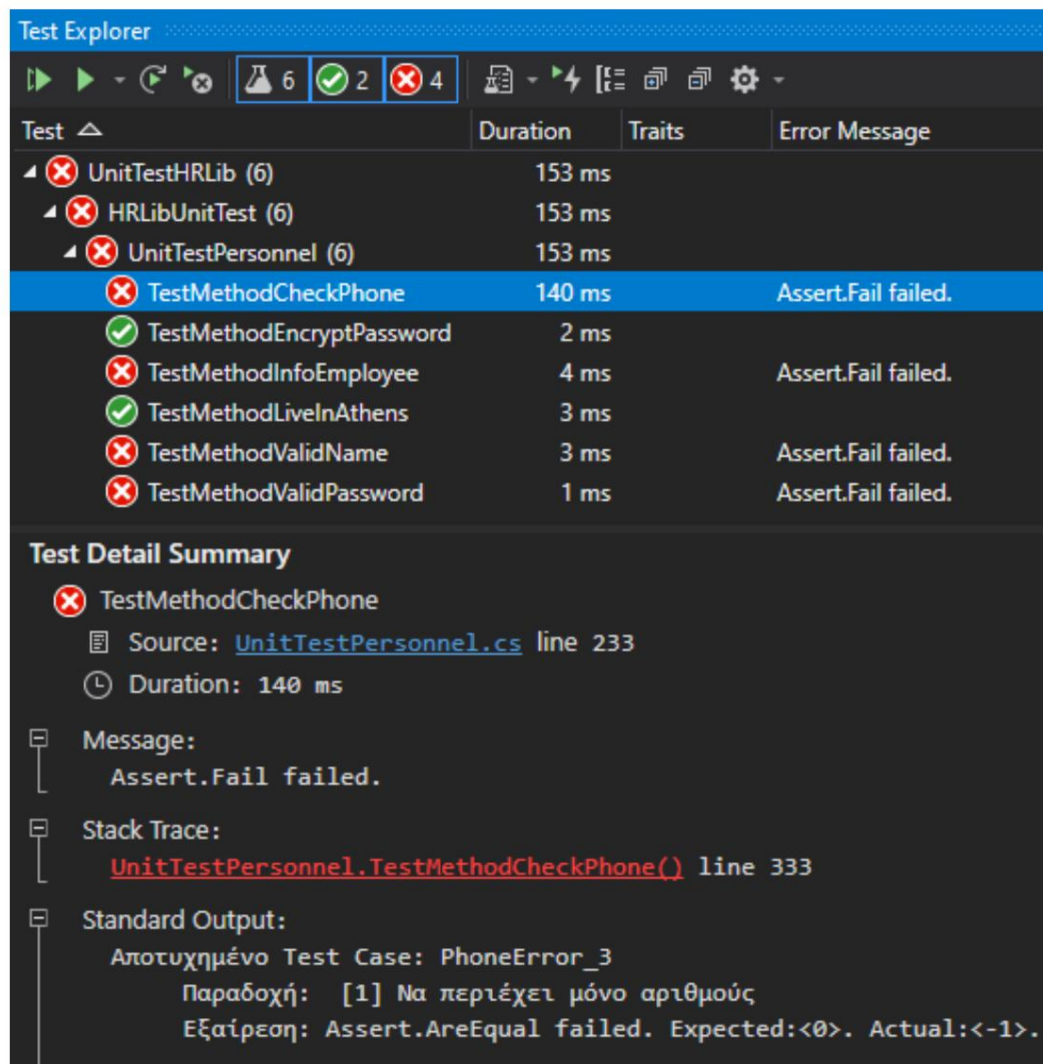
According to the implementation assumptions of the `CheckPhone()` method mentioned [here](#), the input data with #id PhoneError_3, violates the assumption:

[1] Contain only numbers

The phone number in the test case with #id PhoneError_3 does not contain 10 characters, as there are spaces ' ' between the digits, which leads to an error. Essentially, the software prevents the entry of any other character in the phone number besides digits, even though the phone number 210 28 12 967 is an acceptable landline phone in the Athens - Piraeus Metropolitan Area zone. This poses a problem for the reliability of the software, as the different syntax of the logically correct element rejects its verification.

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

3.5. Screenshot of Test Explorer



4. Employee Age Error

4.1. Error ID

AgeError_4

4.2. Name of failed control method

TestMethodInfoEmployee

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

4.3. Description of the audit carried out

The check is performed in order to calculate the age and years of service of an employee. In detail, if the dates meet all the implementation assumptions of the InfoEmployee() method mentioned [here](#), then they correspond to valid dates of birth and hire, respectively. Otherwise, if at least one assumption is violated by at least one date, then the age and years of service of the employee are not calculated and therefore their dates are considered invalid for the requirements of the software.

4.4. Report an Error that occurred

Employee	Name	Home Phone	Mobile Phone	Birthday	Hiring Date
emplfault1	Aggelos Skandalis 2501010102		6988558589	1953-05-30	1977-02-03

El.	Test Input Data	Expected Results		
#id	Employee	Age	Years Of Experience 47	Hint
AgeError_4	emplFault1	70		ageBetween18And70

Hint	Message
ageBetween18And70	[1] Age must be between 18-70 years old

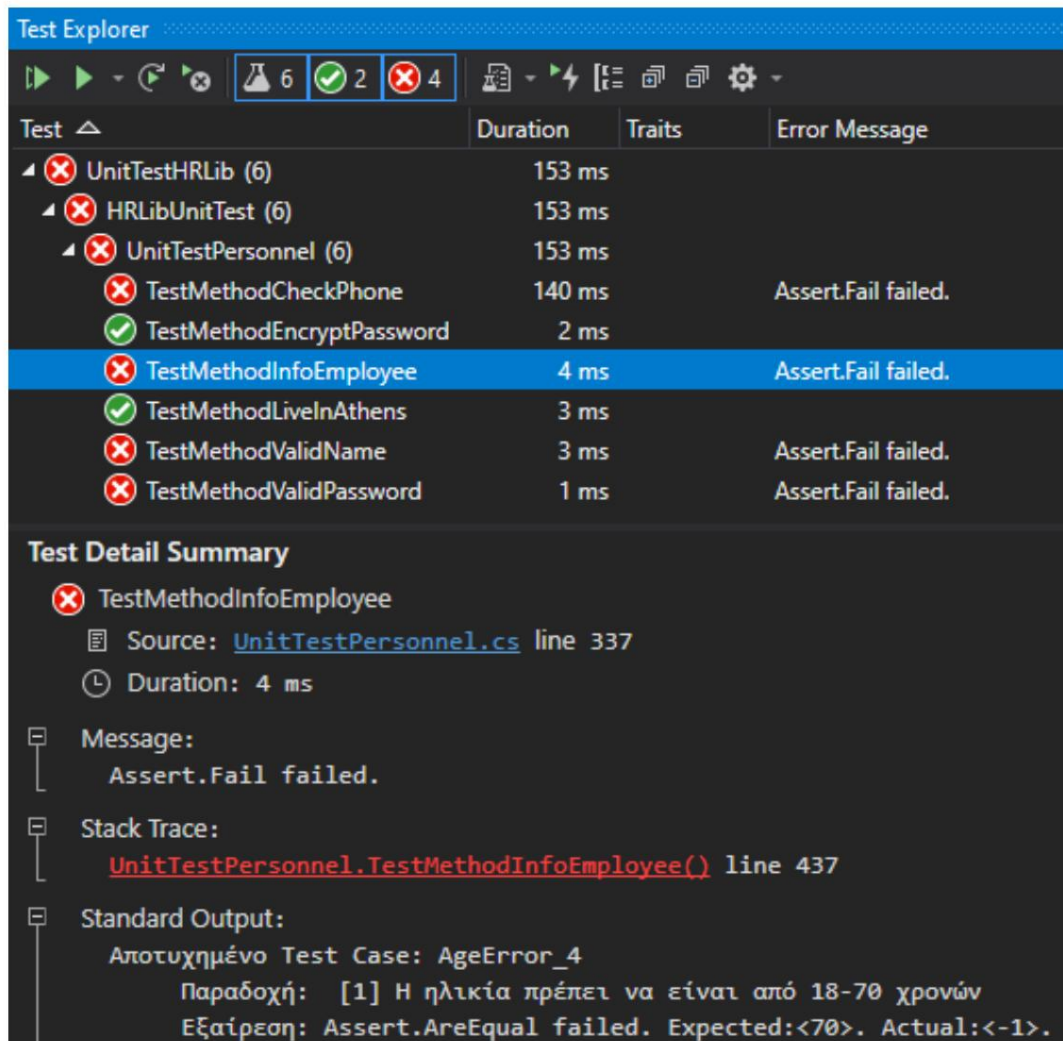
According to the implementation assumptions of the InfoEmployee() method mentioned [here](#), the input data with #id AgeError_4, violates the assumption:

[1] Age must be between 18-70 years old

The birth date of the test case with #id AgeError_4 corresponds to an employee who is over 70 years old and therefore does not meet the software requirements to be retained in the company's data.

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

4.5. Screenshot of Test Explorer



5. Error in an Employee's Years of Service

5.1. Error ID

YearsOfXpError_5

5.2. Name of failed control method

TestMethodInfoEmployee

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

5.3. Description of the audit carried out

The check is performed in order to calculate the age and years of service of an employee. In detail, if the dates meet all the implementation assumptions of the InfoEmployee() method mentioned [here](#), then they correspond to valid dates of birth and hire, respectively. Otherwise, if at least one assumption is violated by at least one date, then the age and years of service of the employee are not calculated and therefore their dates are considered invalid for the requirements of the software.

5.4. Report an Error that occurred

Employee	Name	Home Phone	Mobile Phone	Birthday	Hiring Date
emplfault2	Dimitris Siametis 21	104588989	6999999999	2006-01-02	2024-01-01

El.	Test Input Data	Expected Results		
#id	Employee	Age	Years Of Experience 0	Hint
YearsOfXpError_5	emplFault2	18		youngerThan18ForHiring

Hint	Message
youngerThan18ForHiring [2]	Hiring date must be from date of birth later by 18 years to the current date

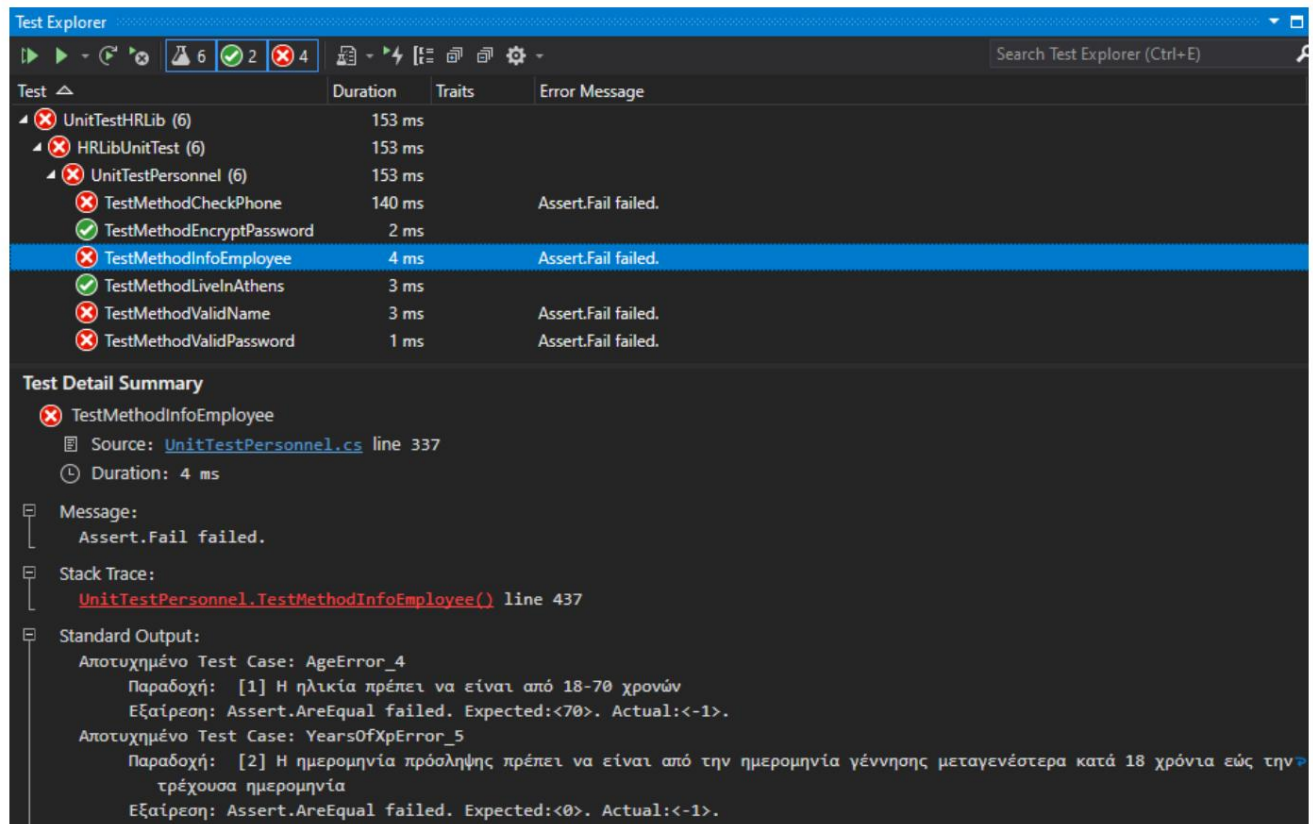
According to the implementation assumptions of the InfoEmployee() method mentioned [here](#), the input data with #id YearsOfXpError_5, violates the assumption:

[2] Date of hire must be from date of birth plus 18 years to current date

The hiring date of the test case with #id YearsOfXpError_5 corresponds to an employee who has not reached the age of 18 and therefore does not meet the software requirements to be retained in the company's data.

ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

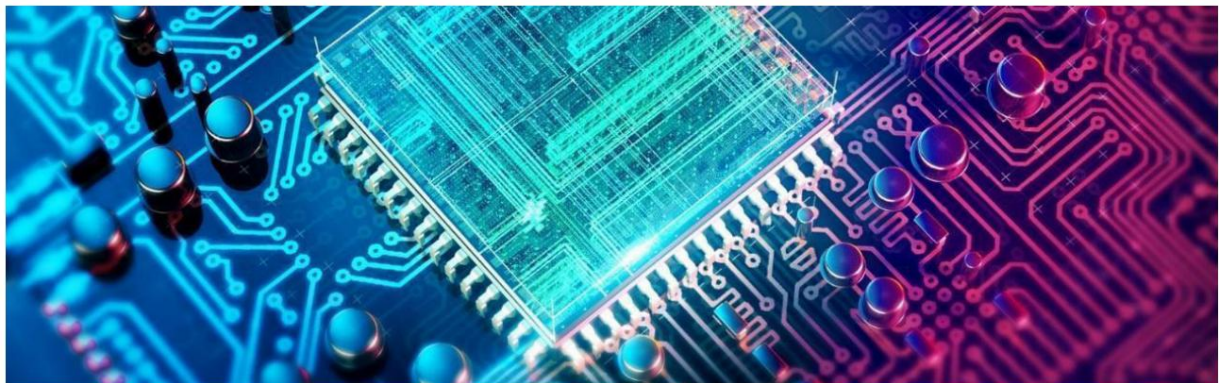
5.5. Screenshot of Test Explorer



ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ



Thank you for your attention.



ΠΟΙΟΤΗΤΑ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΛΟΓΙΣΜΙΚΟΥ

